



TheLLMEngineer ✓

LangGraph

with

Agentic AI

(Complete Notes)



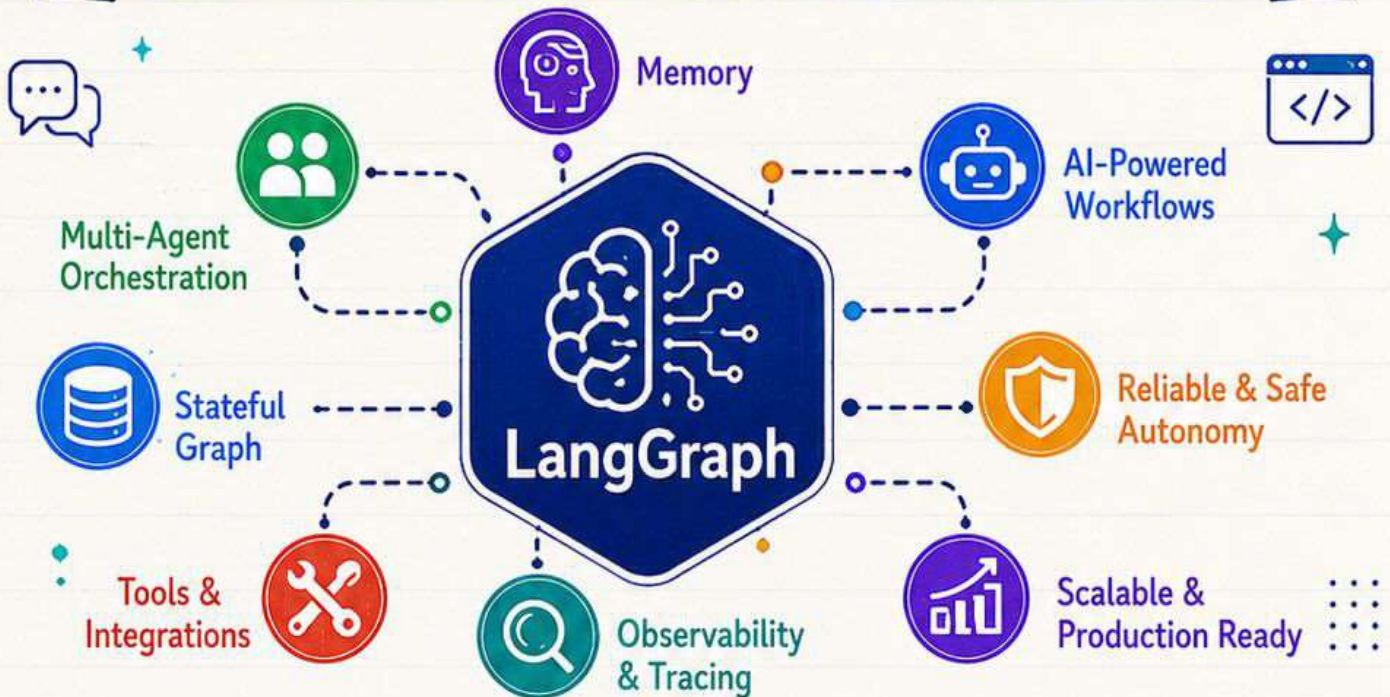
With Syllabus

&



Top 50

Interview Questions (Easy to Advanced)



Learn
Faster



Build
Smarter



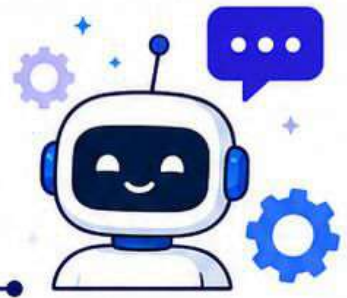
Deploy
Confidently



Scale
Effortlessly

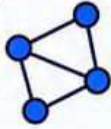


LangGraph with Agentic AI



SYLLABUS

Module 1: LangGraph Fundamentals



- What is LangGraph?
- LangGraph vs LangChain
- When to use LangGraph
- Stateful workflows
- Graph-based execution model

Module 2: Core Components



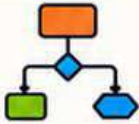
- State
- StateGraph
- MessageGraph
- Nodes
- Edges
- START and END nodes
- Graph compilation
- Graph execution ('invoke', 'stream')

Module 3: State Management



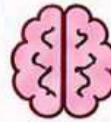
- TypedDict State
- Pydantic State
- State updates
- Reducers
- Shared state
- Multiple state schemas
- State persistence

Module 4: Graph Control Flow



- Sequential execution
- Conditional Edges
- Dynamic Routing
- Loops
- Parallel branches
- Error handling

Module 5: Memory



- Short-term memory
- Long-term memory
- Conversation history
- Checkpointing
- Persistent memory

Module 6: Human-in-the-Loop



- Interrupts
- Resume execution
- Approval workflows
- Manual state editing

Module 7: Tool Calling



- Creating tools
- ToolNode
- Tool execution
- Multi-tool workflows
- Error recovery

Module 8: Multi-Agent Systems



- Agent architecture
- Supervisor pattern
- Router pattern
- Planner-Executor pattern
- Multi-agent collaboration

Module 9: Streaming & Execution



- Token streaming
- Event streaming
- Async execution
- Sync execution
- Runtime monitoring

Module 10: Persistence



- Checkpointers
- SQLite checkpointer
- PostgreSQL checkpointer
- Recovery after failures
- Durable execution

Module 11: Debugging & Observability



- Graph visualization
- Execution tracing
- Logging
- Debugging nodes
- LangSmith integration

Module 12: Production Deployment



- API deployment
- Scaling
- Performance optimization
- Security
- Best practices



Build Intelligent
Agentic Workflows



Leverage LLMs
Effectively



Reliable, Scalable
& Production Ready



Optimize, Monitor
& Improve



Collaborate with
Multi-Agent Systems

* LangGraph with Agentic AI *

Module 1 : LangGraph Fundamentals *

1. What is LangGraph ?

LangGraph is a library for building stateful, reliable and controllable AI agents using graphs. It is built on top of LangChain but adds structure, memory and control flow to create advanced agentic workflows.

Key Point

- ✓ Stateful
- ✓ Graph based
- ✓ Reliable execution
- ✓ Human-in-the-loop
- ✓ Built for complex agents

2. LangGraph vs LangChain

LangChain	LangGraph
• Chain based (Linear flow) • Stateless by default • Good for simple apps • Limited control flow • Hard to manage complex workflows	• Graph based (Flexible flow) • Stateful by design • Best for complex agents • Advanced control flow • Built-in memory & persistence

3. When to use LangGraph ?

- When your agent has multiple steps and decisions.
- When you need memory and state management.
- When you want human-in-the-loop or approval steps.
- When building multi-agent systems.
- When reliability and persistence are important.

4. Stateful Workflows

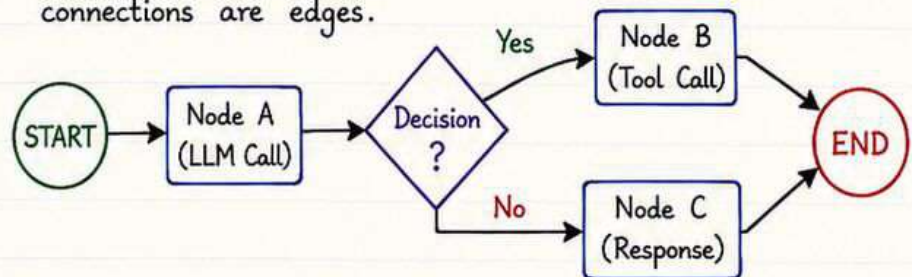
LangGraph allows maintaining state across steps. The state is passed, updated and used by different nodes in the graph.

Example State can hold :

- Messages
- User data
- Intermediate results
- Tool outputs

5. Graph-based Execution Model

In LangGraph, the workflow is represented as a graph. Each step is a node and the connections are edges.



Summary

LangGraph gives structure, memory and control to AI agents. It helps in building powerful, reliable and production-ready agentic workflows.

LangGraph with Agentic AI

Module 2 : Core Components of LangGraph

① State

State is the shared data that flows throughout the graph. It stores all the information needed by different nodes to perform their tasks.

Key Points :

- State is mutable and updated at each step.
- Defined using TypedDict or Pydantic BaseModel.
- Acts as the memory of the graph.

Example (TypedDict) :

```
from typing import TypedDict, List
class MyState(TypedDict):
    messages: List[dict]
    user_input: str
    result: str
```

② StateGraph

StateGraph is the main graph class that orchestrates the workflow. It consists of nodes connected by edges and operates on the shared state.

Key Points :

- Manages state flow between nodes.
- Supports conditional and parallel flows.
- Compiled before execution.
- Provides `invoke()` and `stream()` methods.

Example :

```
from langgraph.graph import StateGraph, END
workflow = StateGraph(MyState)
# Add nodes and edges
workflow = workflow.add_node("node_a", func_a)
workflow = workflow.add_node("node_b", func_b)
workflow = workflow.set_entry_point("node_a")
workflow = workflow.add_edge("node_a", "node_b")
workflow = workflow.add_edge("node_b", END)
app = workflow.compile()
```

③ MessageGraph

MessageGraph is a special graph optimized for conversation-based AI agents. It works with message objects (like chat history) and simplifies handling of dialogues.

Key Points :

- Built for chat/message based workflows.
- Uses messages as the primary state.
- Automatically handles message history.
- Ideal for LLM chat agent pipelines.

Example Use Case :

Chatbot, Customer support agent, Assistant with conversation memory.

④ Nodes

Nodes are the individual processing units in a LangGraph. Each node performs a specific task or operation on the state.

Key Points :

- Represent a function or tool.
- Takes state as input and returns updated state.
- Can call LLMs, tools, APIs, or any logic.

Example :

```
def node_a(state: MyState) -> MyState:
    state["result"] = "Processed by A"
    return state
```

⑤ Edges

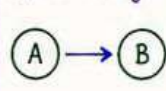
Edges define the flow between nodes. They determine the order of execution and how control moves from one node to another.

Types of Edges :

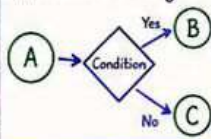
- Normal Edge : Fixed connection between two nodes.
- Conditional Edge : Next node depends on a condition.
- Parallel Edge : Multiple paths can be executed.
- Loop Edge : Allows cycles in the graph.

Examples with Diagrams :

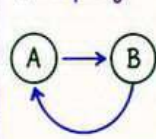
(a) Normal Edge



(b) Conditional Edge



(d) Loop Edge



Quick Summary

State → Data/Memory
StateGraph → Orchestrator
MessageGraph → For chat workflows
Nodes → Do the work
Edges → Define the flow



Takeaway : Visualization helps you see, tracing helps you follow, and logging helps you know what's happening.



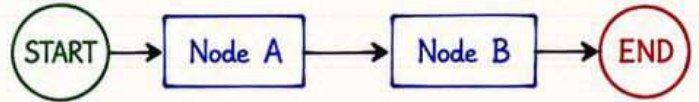
* LangGraph with Agentic AI *

Module 2 (Part 2) : Core Components of LangGraph

① START and END nodes

- START is the entry point of the graph. It is a virtual node where execution begins.
- END is the terminal node. When the graph reaches END, the execution stops.
- We don't add START and END manually. LangGraph provides them automatically.

Diagram :



Key Points :

- START has no incoming edge.
- END has no outgoing edge.
- Multiple paths can lead to END.
- The graph must have at least one path from START to END.

② Graph Compilation

After building the graph (adding nodes and edges), we need to compile it before execution.

Compilation performs :

- Validates the graph (checks for errors)
- Optimizes internal structures
- Prepares the graph for execution

Example :

```
workflow = StateGraph(MyState)
workflow.add_node("node_a", func_a)
workflow.add_node("node_b", func_b)
workflow.add_edge(START, "node_a")
workflow.add_edge("node_a", "node_b")
workflow.add_edge("node_b", END)
app = workflow.compile()
# Now the graph is ready to run
```

Syntax :

```
from langgraph.graph import StateGraph
workflow = StateGraph(MyState)
# add nodes and edges
...
app = workflow.compile() # Compiles the graph
```

Note :

Without compilation, you cannot run the graph. Think of compilation like building an app before you run it.

③ Graph Execution (invoke, stream)

Once compiled, we can execute the graph. LangGraph provides two main execution methods.

A. invoke()

- Runs the graph synchronously.
- Waits for the full execution and returns the final output/state.
- Best for simple use cases.

Syntax :

```
result = app.invoke(initial_state)
```

Example :

```
initial_state = {"messages": [{"role": "user", "content": "Hi"}]}
result = app.invoke(initial_state)
print(result)
# returns final state after execution
```

B. stream()

- Runs the graph and yields intermediate outputs (step by step).
- Useful for real-time updates, streaming responses, or long workflows.

Syntax :

```
for output in app.stream(initial_state):
    print(output)
```

Example :

```
for step_output in app.stream(initial_state):
    print("Step Output:", step_output)
# yields state after each node/step
```

When to use ?

- Use invoke() when you need the final result only.
- Use stream() when you want step-by-step updates or real-time output.

Flow : Build Graph → Compile Graph → Execute Graph (invoke / stream)

LangGraph with Agentic AI *

Module 3 (Part 1) : State Management in LangGraph

① TypedDict State

TypedDict is a lightweight way to define the structure of the state using Python's typing module.

Key Points :

- Simple and fast.
- No runtime validation.
- Ideal for basic and internal use.

Example :

```
from typing import TypedDict, List
class MySants(TypedDict):
    messages : List[dict]
    user_input : str
    steps : int
    completed : bool
```

Usage in LangGraph :

Used as the state schema when strict runtime validation is not required.

② Pydantic State

Pydantic provides data validation and type enforcement at runtime.

Key Points :

- Validates data at runtime.
- Provides default values and field constraints.
- Best for production and external inputs.

Example :

```
from pydantic import BaseModel, Field
class MySants(BaseModel):
    messages : List[dict] = Field(default_factory=list)
    user_input : str
    steps : int = 0
    completed : bool = False
    class Config:
        extra = "forbid" # no extra fields allowed
```

Uden you sater med validation, default values.

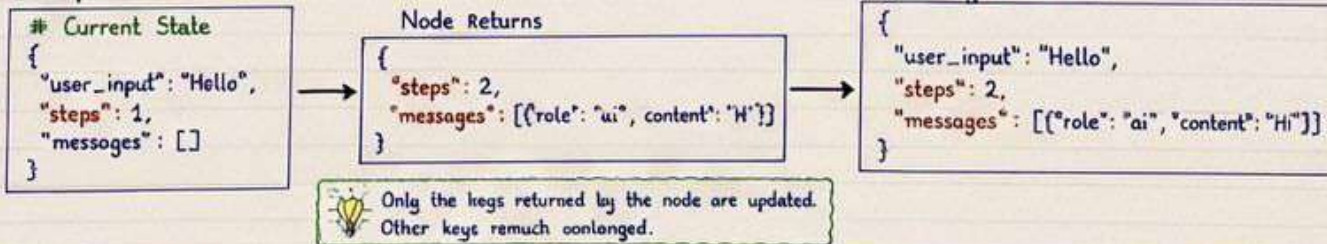
③ State Updates

State updates occur when a node returns a partial state. LangGraph merges this update into the current state.

Key Points :

- Nodes return only the changes.
- LangGraph merges the changes into the full state.
- Updates are immutable (create new state internally).
- Only the returned keys are updated.

Example :



④ Reducers

Reducers decide how to combine previous state and new updates for specific keys.

They are useful for managing complex or cumulative data.

Key Points :

- Applied per-key.
- Useful for lists, sets, dicts, etc.
- Define custom logic for merging.

Common Reducers :

Reducer	Use Case
add_messages	Append new messages to list
operator add	Increment counters / reducers
set_union	Union of sets
custom function	Custom merge logic

Example : (Using add_messages reducer)

```
from langgraph.ppt import Annotated, add_messages
from typing import Annotated, List, TypedDict

class MyState(TypedDict):
    messages : Annotated[List, add_messages]
    steps : int
```

What it does ?

The add_messages reducer automatically appends new messages to the existing list instead of overwriting it.

Existing State

```
messages = [
    {"role": "user",
     "content": "Hi"}
]
```

New Update

```
messages = [
    {"role": "ai",
     "content": "Hello"}
]
```

After Reducer

```
messages = [
    {"role": "user",
     "content": "Hi"},
    {"role": "ai",
     "content": "Hello"}
]
```

Merged using
add_messages
reducer

* LangGraph with Agentic AI *

Module 3 (Part 2) ;

State Management in LangGraph

① Shared State

Shared state is accessible and modifiable by all nodes in the graph.

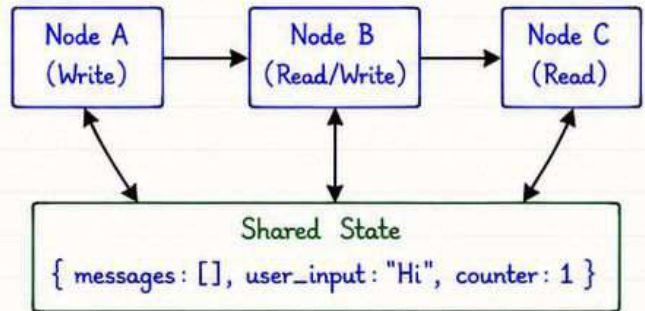
Key Points :

- All nodes read/write the same state object.
- Changes in one node are visible to others.
- Enables coordination and data sharing across the workflow.

Example :

```
from typing import TypedDict
class MyState(TypedDict, total=False):
    messages: list
    user_input: str
    counter: int
```

Diagram :



Use Case :

When multiple nodes need to collaborate using the same data (e.g., collect info, summarize, respond).

② Multiple State Schemas

LangGraph allows using different state schemas for different parts (or subgraphs) of the workflow.

Key Points :

- Each subgraph or node group can have its own state schema.
- Useful for modular and complex applications.
- Helps in type safety and separation of concerns.

Example :

Schema 1 (Input Collection)

```
class InputState(TypedDict):
    name: str
    email: str
    query: str
```

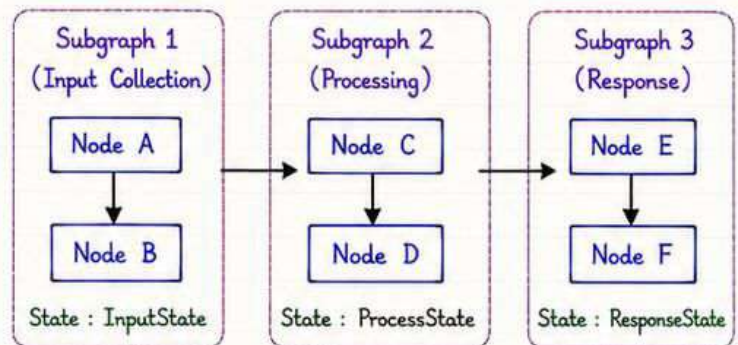
Schema 2 (Processing)

```
class ProcessState(TypedDict):
    query: str
    context: list
    answer: str
```

Schema 3 (Response)

```
class ResponseState(TypedDict):
    answer: str
    sources: list
    confidence: float
```

Diagram (Subgraphs with Different States) :



Use Case :

Large applications where different stages need different data structures. Example: Data collection → Processing → Response generation.

③ State Persistence

State persistence allows saving the state so that it can be restored later (even after program restart).

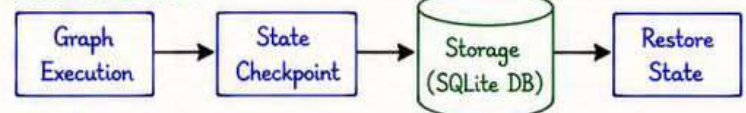
Key Points :

- Saves state to durable storage.
- Enables fault tolerance and recovery.
- Supports long-running workflows.
- Can be combined with checkpointing.

Example (Using SQLite Checkpointer) :

```
from langgraph.checkpoint.sqlite import SqliteSaver
checkpointer = SqliteSaver.from_conn_string("checkpoints.db")
app = workflow.compile(checkpointer=checkpointer)
# State will be saved automatically during execution
```

How it works :



Use Case :

- Resume a workflow after a failure.
- Maintain user conversation across sessions.
- Save long running agent states.



Note :

State persistence is essential for production grade agentic applications to ensure reliability and data durability.

* LangGraph with Agentic AI *

Module 4 : Graph Control Flow (Part 1)

Graph Control Flow defines how execution moves between nodes in a LangGraph.

--- How the Graph Decides the Flow ---

Key Takeaways

- Control flow makes the workflow flexible.
- Use conditions and routing for smart decisions.
- Helps in building real-world agentic workflows.

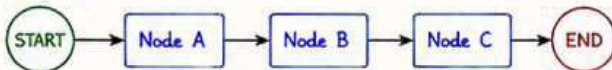
① Sequential Execution

In sequential execution, nodes are executed one after another in a fixed order.

Key Points :

- Simple and predictable flow.
- Every node runs once in the defined order.
- Good for linear workflows.

Diagram :



Example Use Case :

Data cleaning → Data processing → Result generation

Example (Sequential Graph)

```
from langgraph.graph import StateGraph, END
from typing import TypedDict

class MyState(TypedDict):
    messages: str

workflow = StateGraph(MyState)
workflow.add_node("step1", func_step1)
workflow.add_node("step2", func_step2)
workflow.add_node("step3", func_step3)
workflow.add_edge(START, "step1")
workflow.add_edge("step1", "step2")
workflow.add_edge("step2", "step3")
workflow.add_edge("step3", END)
app = workflow.compile()
# Execution order: step1 → step2 → step3
```

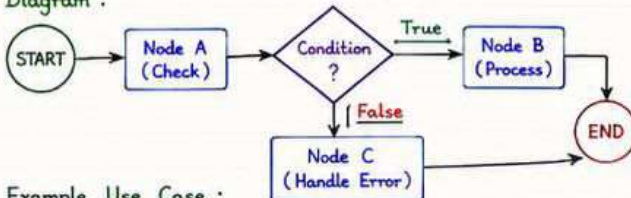
② Conditional Edges

Conditional edges allow the graph to branch based on a condition (True / False or any logic).

Key Points :

- Adds decision making to the workflow.
- Next node depends on the outcome.
- Useful for validation, checks, approvals, etc.

Diagram :



Example Use Case :

Validate user input → If valid go ahead → If not, show error and stop.

Example (Conditional Edges)

```
def check_input(state: MyState) → str:
    if "ok" in state["message"].lower():
        return "valid"
    else:
        return "invalid"

workflow = StateGraph(MyState)
workflow.add_node("check", check_input)
workflow.add_node("ok", func_ok)
workflow.add_node("error", func_error)
workflow.add_edge(START, "check")
workflow.add_conditional_edges(
    "check", check_input, {"valid": "ok", "invalid": "error"}
)
workflow.add_edge("ok", END)
workflow.add_edge("error", END)
```

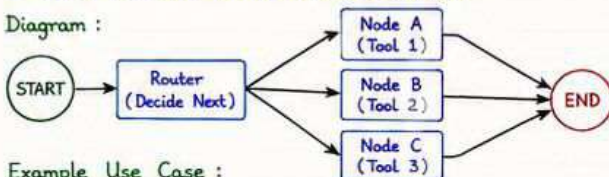
③ Dynamic Routing

Dynamic routing allows selecting the next node from multiple options based on runtime logic or data.

Key Points :

- More flexible than simple conditions.
- Next node is chosen dynamically from many.
- Useful for multi-intent, tool selection, etc.

Diagram :



Example Use Case :

User asks a question → Router chooses tool (search / calculator / database) based on intent.

Example (Dynamic Routing)

```
def router(state: MyState) → str:
    msg = state["message"].lower()
    if "search" in msg:
        return "search_node"
    elif "calc" in msg:
        return "calculator_node"
    else:
        return "db_node"

workflow = StateGraph(MyState)
workflow.add_node("router", router)
workflow.add_node("search_node", func_search)
workflow.add_node("calculator_node", func_calc)
workflow.add_node("db_node", func_db)
workflow.add_edge(START, "router")
workflow.add_conditional_edges(
    "router", router, {"search_node": "search_node",
                       "calculator_node": "calculator_node",
                       "db_node": "db_node"}
)
workflow.add_edge("search_node", END)
workflow.add_edge("calculator_node", END)
workflow.add_edge("db_node", END)
```



Summary :

- ✓ Sequential Execution → Fixed order
- ✓ Conditional Edges → Two or more paths based on condition
- ✓ Dynamic Routing → Many possible next nodes based on logic/data

Remember :

Good control flow = Smart agentic behavior
Better decisions = Better AI workflows



* LangGraph with Agentic AI *

Module 4 (Part 2) :

Graph Control Flow (Part 2)

① Loops

Loops allow the graph to repeat a set of nodes until a condition becomes **False**.

Key Points :

- Useful for iterative tasks and retries.
- Continue looping while the condition is **True**.
- Must have an exit condition to avoid infinite loops.

Example : (Retry until success)

```
def should_retry(state):  
    return state["retry_count"] < 3 and not state["success"]
```

Flow : Task → Check → (if retry) → Task → Check → ... → END

Loop Diagram :

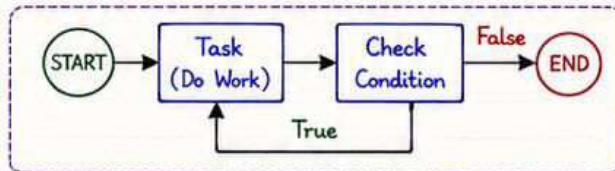
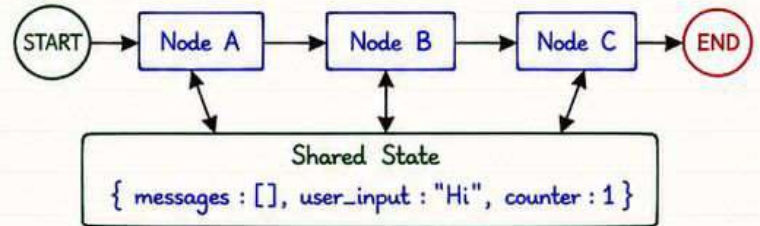


Diagram :



Key Points :

- START has no incoming edge.
- END has no outgoing edge.
- Multiple paths can lead to END.
- The graph must have at least one path from START to END.

Use Case :

When multiple nodes need to collaborate using the same data (e.g., collect info, summarize, respond).

Always ensure a clear exit condition to prevent infinite loops.

② Parallel Branches

Parallel branches allow multiple paths to run concurrently and merge results later.

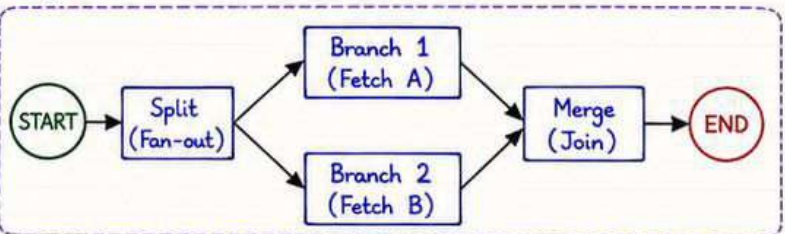
Key Points :

- Split the flow into independent branches.
- All branches can run at the same time.
- Results are merged at a join node.
- Improves speed and efficiency.

Example : (Fetch data from multiple sources)

```
# Split → FetchA & FetchB run in parallel → Merge → END  
# (Pseudocode)  
result_a = fetch_source_a() # branch 1  
result_b = fetch_source_b() # branch 2  
final = merge(result_a, result_b)
```

Diagram :



Use Case :

- Fetching data from multiple APIs
- Parallel tool calls
- Multi-document retrieval in RAG

Branches run independently, final result is merged.

③ Error Handling

Error handling ensures the graph can gracefully handle failures without crashing.

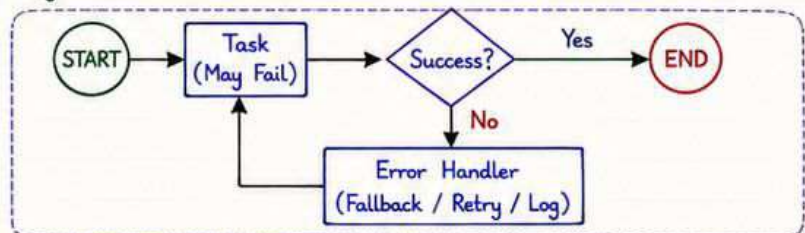
Key Points :

- Catch exceptions in nodes.
- Route to fallback or recovery nodes.
- Log the error and continue or end safely.

Example :

```
try:  
    result = risky_task(state)  
except Exception as e:  
    state["error"] = str(e)  
    return {"error": True} # go to Error Node
```

Diagram :



Use Case :

- API failures, timeouts
- Invalid inputs
- Rate limit / quota exceeded

Good error handling = Resilient & production-ready agent workflows.



Summary : Loops repeat work | Parallel branches speed up execution
Error handling makes the workflow robust and reliable.

LangGraph with Agentic AI

Module 5 : Memory

Managing Memory in LangGraph

① Short-term Memory

Short-term memory holds information during a single run or session of the graph.

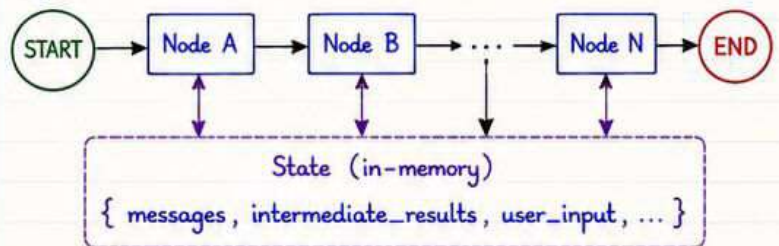
Key Points :

- Stored in the state object.
- Available to all nodes in the current execution.
- Lost after the execution ends (unless saved).

Use Case :

- Temporary data needed within a single workflow run (e.g., tool results, partial answers).

Diagram :



② Long-term Memory

Long-term memory stores information across multiple sessions or users.

Key Points :

- Stored in external storage (DB, vector store, etc.).
- Can be retrieved in future runs.
- Useful for personalized and scalable agents.

Diagram :



Use Case :

- Remember user preferences, past interactions, documents, or knowledge across sessions.

③ Conversation History

Conversation history keeps the record of all messages exchanged in a conversation.

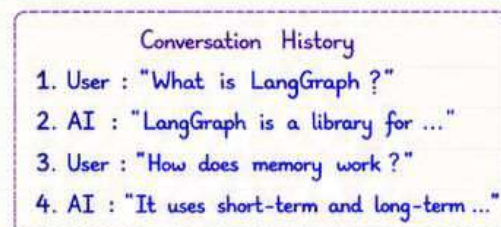
Key Points :

- Typically a list of messages (role + content).
- Sent to LLM in each step for context.
- Can be stored in short-term or persisted.

Use Case :

- Chatbots, assistants, support agents.
- (Maintains context for natural conversations)

Diagram :



④ Checkpointing

Checkpointing saves the state of the graph at certain points so it can be resumed later.

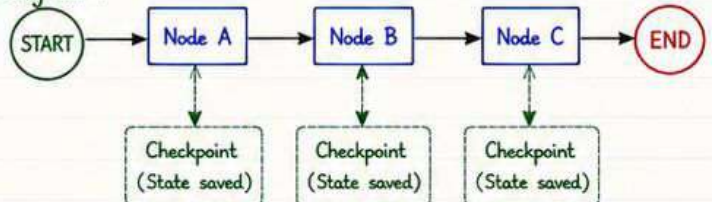
Key Points :

- Saves state after each step or at defined points.
- Enables recovery from failures or interruptions.
- Works with Checkpointer implementations.

Use Case :

- Long-running workflows, reliability, fault tolerance.

Diagram :



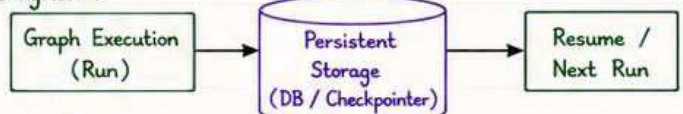
⑤ Persistent Memory

Persistent memory combines long-term storage with checkpointing to make memory durable.

Key Points :

- State and history are saved to persistent storage.
- Survives restarts, crashes, and deployments.
- Often implemented using DBs (SQLite, Postgres, etc.).

Diagram :



Use Case :

- Production-grade agents that must remember state and conversations across time.



Takeaway :

Memory in LangGraph enables agents to interact intelligently by remembering context now (short-term) and knowledge forever (long-term). Checkpointing and persistence make the agents reliable and production-ready.



LangGraph with Agentic AI

Module 6 : Human-in-the-Loop

Putting Humans in Control When Needed

① Interrupts

Interrupts pause the graph execution at specific points and wait for human input or decision.

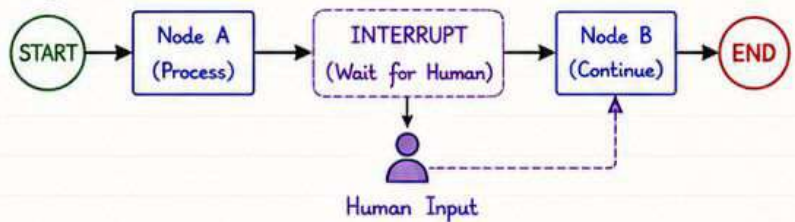
Key Points :

- Triggered by a node or condition.
- Execution pauses and control goes to the human.
- State is saved at the interrupt point.
- Enables safe and controlled workflows.

Use Case :

- When an action needs human review before proceeding (e.g., high-risk operations).

Diagram :



Use Case Example :

Before sending an important email or charging a user, pause and ask for human confirmation.

② Resume Execution

After human input is provided, the graph resumes from where it was interrupted.

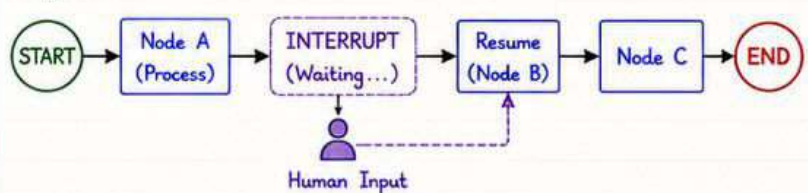
Key Points :

- Resume continues from the saved state.
- Human input is injected into the state.
- Workflow completes the remaining steps.

Use Case :

- Continue processing after approval or additional information.

Diagram :



Use Case Example :

After a manager approves a discount, resume the flow to apply discount and send the invoice.

③ Approval Workflows

Approval workflows ensure that humans review and approve (or reject) certain actions or decisions.

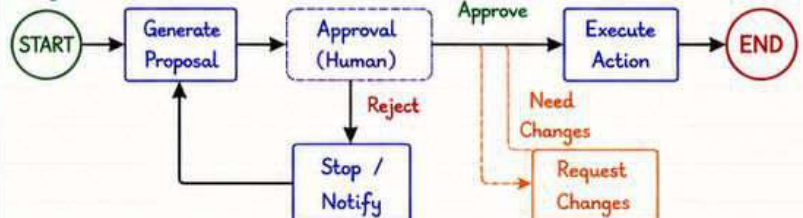
Key Points :

- Request approval from one or more approvers.
- Can be approve / reject / request changes.
- Based on the decision, the graph follows different paths.

Use Case :

- Budget approval, content publishing, user verification, compliance checks.

Diagram :



Use Case Example :

A marketing campaign is created, manager approves it, then it is published. If rejected, the flow stops or asks for changes.

④ Manual State Editing

Humans can manually view and edit the state during a pause to fix issues or add information.

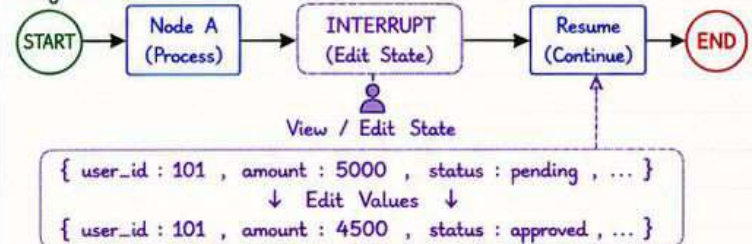
Key Points :

- Inspect current state variables.
- Edit values manually.
- Save changes and resume execution.
- Useful for debugging or correcting data.

Use Case :

- Correct wrong data, fill missing info, or override LLM output.

Diagram :



Use Case Example :

An agent extracted wrong amount from a document. Human edits the amount in the state and resumes the flow.



Takeaway : Human-in-the-loop makes LangGraph agents safer, more reliable, and suitable for real-world business processes.

* Lang Graph with Agentic AI *

Module 7 : Tool Calling

Empowering Agents to Use Tools

① Creating Tools

Tools are external functions or APIs that agents can use to get information or perform actions.

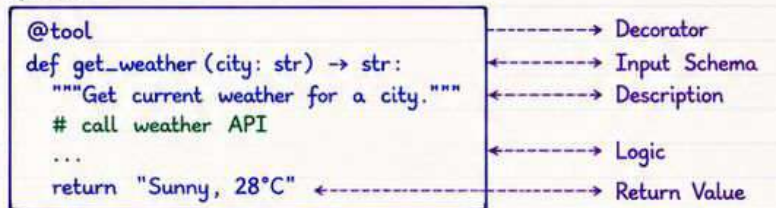
Key Points :

- Tools wrap any function or API.
- Define input schema and description.
- Return results as structured output.
- Tools help agents interact with the real world.

Use Case :

- Search the web, get current weather, calculate math, send email, etc.

Diagram :



Use Case Example :

Create a tool to fetch stock price of a company using Finance API.

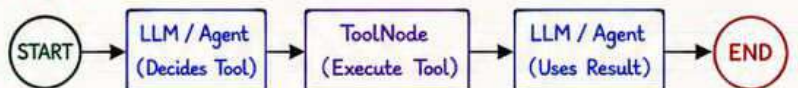
② ToolNode

ToolNode is a special node that executes a tool requested by the agent.

Key Points :

- Receives tool call from the LLM.
- Executes the requested tool.
- Returns the result back to the agent.
- Handles single or multiple tool calls.

Diagram :



Use Case Example :

Agent asks for the current weather.
ToolNode calls `get_weather()` and returns the weather info.

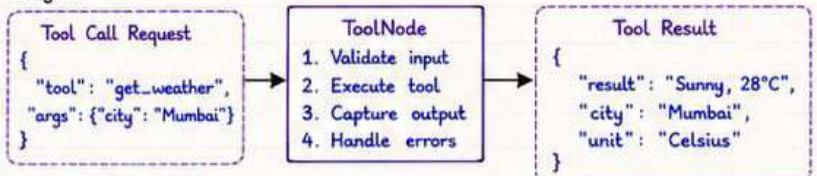
③ Tool Execution

ToolNode executes tools safely and returns the output to the agent for reasoning.

Key Points :

- Validates tool input.
- Executes tool and captures output.
- Handles exceptions and timeouts.
- Returns structured result to the graph state.

Diagram :



Use Case Example :

Execute calculator tool to add 45 + 67 and return the result.

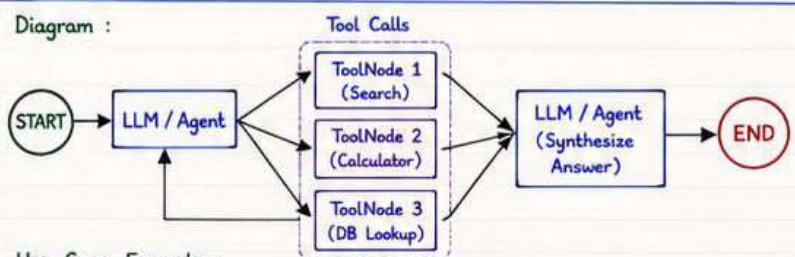
④ Multi-tool Workflows

Agents can use multiple tools in a single workflow based on the task.

Key Points :

- LLM decides which tool to call next.
- Multiple tool calls can happen in sequence or in parallel.
- Results from tools are combined for final answer.

Diagram :



Use Case Example :

Search news → Get data → Calculate summary → Generate final report.

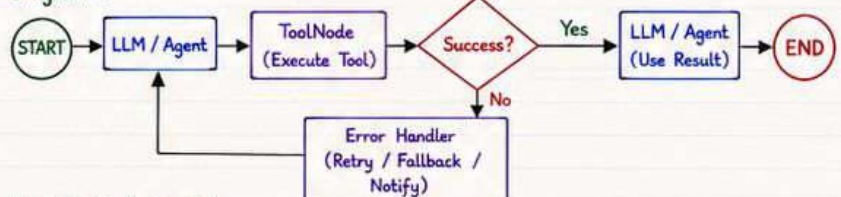
⑤ Error Recovery

Tools may fail. The graph should recover and continue gracefully.

Key Points :

- Catch tool errors and return friendly message.
- Retry with modified input if needed.
- Fallback to alternative tool.
- Never crash the workflow.

Diagram :



Use Case Example :

If API rate limit error occurs, wait and retry. If still fails, use cached data.



Takeaway : Tool calling enables agents to take real-world actions.
Good tool design + reliable execution = powerful, trustworthy agents!

Lang Graph with Agentic AI

Module 8 : Multi-Agent Systems (Part 1)

Building Teams of Agents that Collaborate

① Agent Architecture

An agent is an autonomous entity that perceives, reasons, acts and communicates to achieve goals.

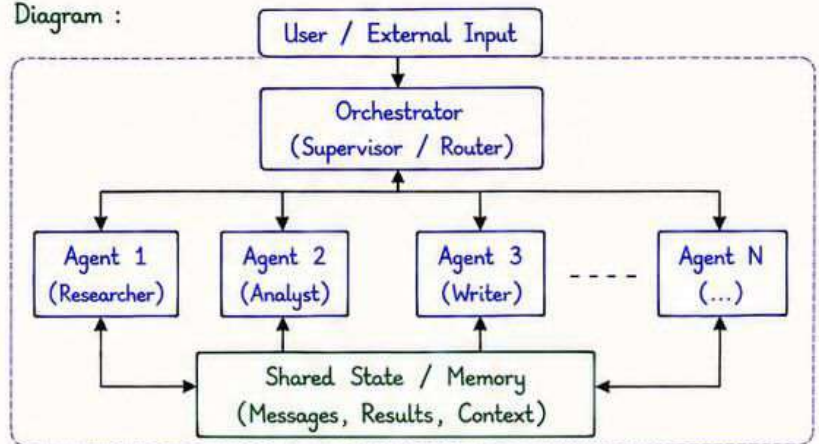
Key Points :

- Each agent has a role, goal and tools.
- Agents can communicate via messages.
- State is shared through the graph.
- Agents can be LLM-powered or rule-based.
- Designed for collaboration and scalability.

Use Case :

- Customer support team with specialized agents (billing, technical, refund, etc.).

Diagram :



Use Case Example :

A research team where one agent finds data, another analyzes it, and another writes the report.

② Supervisor Pattern

A central supervisor agent decides which sub-agent should act next based on the task and state.

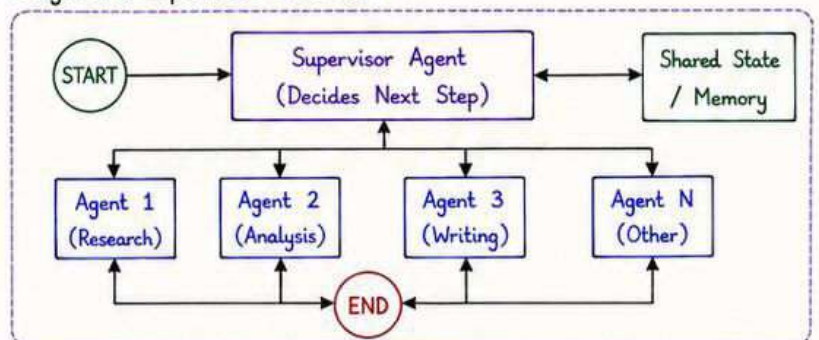
Key Points :

- One supervisor controls the workflow.
- Sub-agents do not talk to each other directly.
- Supervisor delegates tasks and collects results.
- Better for control, safety and traceability.
- Works well when tasks are sequential or structured.

Use Case :

- A project manager agent assigns work to specialist agents step by step.

Diagram (Supervisor Pattern) :



Use Case Example :

A travel planning system where the supervisor decides whether to call the flight agent, hotel agent, or activity agent next.

③ Router Pattern

A router agent (or function) directs the request to the most appropriate agent based on the input.

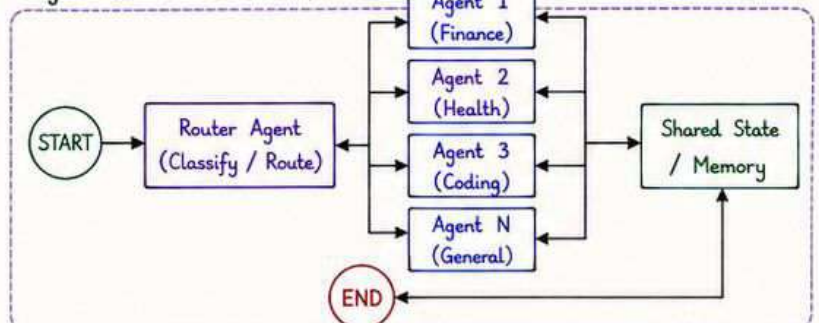
Key Points :

- Router classifies or routes based on intent or context.
- Multiple agents can run in parallel if needed.
- Decentralized after routing, agents may collaborate.
- Good for dynamic, open-ended, multi-domain tasks.
- Often uses LLM classification or embedding similarity.

Use Case :

- An AI assistant that routes user queries to finance, health, coding, or general knowledge agents.

Diagram (Router Pattern)



Use Case Example :

A chatbot that routes "How do I invest?" to the finance agent, "I have a fever" to the health agent, and code errors to the coding agent.



Takeaway : Multi-agent systems allow complex tasks to be solved by a team.
Choose Supervisor Pattern for control, Router Pattern for flexibility.



* LangGraph with Agentic AI *

Module 8 : Multi-Agent Systems (Part 2)

Working Together to Solve Complex Tasks

① Planner-Executor Pattern

In this pattern, one agent (Planner) breaks down the goal into steps. Other agent(s) (Executor) carry out the steps and return results.

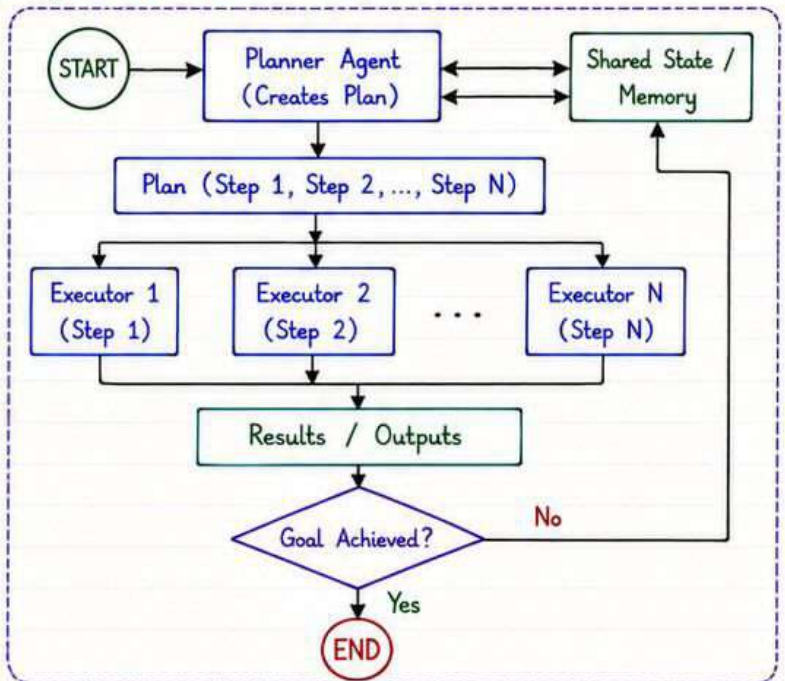
Key Points :

- Planner creates a plan based on the goal.
- Executor agents perform actions or use tools.
- Results are fed back to the planner.
- Iterates until the goal is achieved.
- Great for complex, multi-step tasks.

Use Case :

- Data analysis project where planner creates steps and executor runs code, queries DB, generates charts, etc.

Diagram (Planner-Executor Pattern) :



Use Case Example :

Build a report: Planner creates steps (collect data, analyze, create charts, write summary), Executors run queries, generate charts, and write the report.

② Multi-Agent Collaboration

Multiple agents with different skills work together by sharing information, delegating tasks, and helping each other to achieve a common goal.

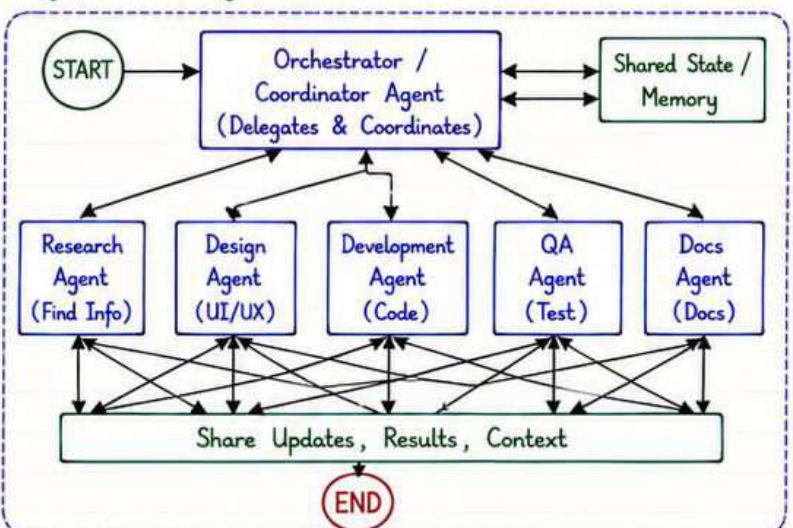
Key Points :

- Agents have specialized roles and expertise.
- They communicate via messages/state.
- Can delegate tasks to the best agent.
- Information is shared to reach better results.
- Supports parallel work and dynamic teamwork.

Use Case :

- Building a product: Research agent gathers info, Design agent creates mockups, Dev agent writes code, QA agent tests it. Docs agent writes documentation.

Diagram (Multi-Agent Collaboration) :



Use Case Example :

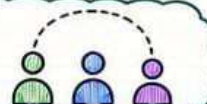
A team builds an app: Research agent collects requirements, Design agent creates UI, Dev agent codes features, QA tests, Docs agent writes documentation. All share info and collaborate.



Takeaway :

Planner-Executor is best for step-by-step problem solving.

Multi-agent collaboration is best for team-based, parallel work with specialized agents.



LangGraph with Agentic AI

Module 9 : Streaming & Execution

Real-time Execution, Better Experience

① Token Streaming

Token streaming sends generated tokens as soon as they are produced.

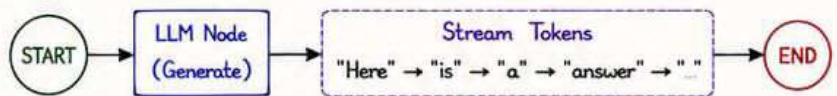
Key Points :

- Improves perceived speed.
- Tokens are streamed incrementally.
- Useful for chatbots and LLM responses.
- Can be displayed in real-time to users.

Use Case :

- Customer support chatbot with instant replies.

Diagram :



Use Case Example :

A chatbot shows the answer word by word instead of waiting for the complete response.

② Event Streaming

Event streaming sends graph events (state updates, node start/end, errors, etc.) in real-time.

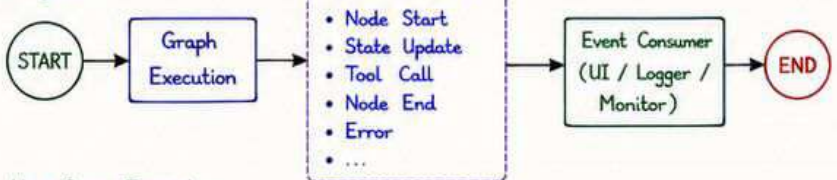
Key Points :

- Helps track what is happening in the graph.
- Events include node start, end, state update, errors, tool calls, etc.
- Useful for dashboards and observability.

Use Case :

- Live monitoring dashboard for running workflows.

Diagram :



Use Case Example :

A Live dashboard shows which node is running, state changes, and any errors in real-time.

③ Async Execution

Async execution runs nodes/tasks without blocking, allowing multiple operations to happen concurrently.

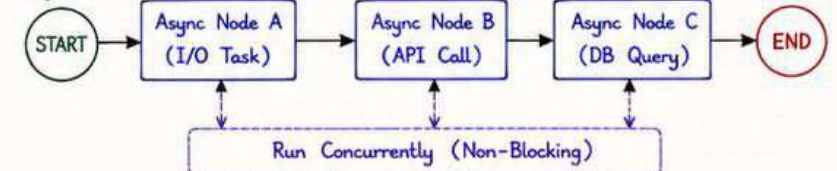
Key Points :

- Uses async/await for non-blocking operations.
- Better performance for I/O bound tasks.
- Allows parallel execution of independent tasks.
- Scales well with many concurrent requests.

Use Case :

- Fetching data from multiple APIs at the same time.

Diagram :



Use Case Example :

Fetching data from multiple APIs at the same time to build a response.

④ Sync Execution

Sync execution runs nodes/tasks one after another in a blocking manner.

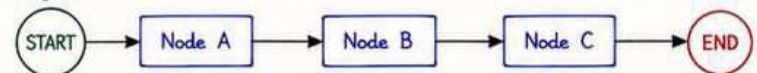
Key Points :

- Simple and easy to debug.
- Each step waits for the previous one.
- Good for CPU bound or dependent tasks.
- Predictable and deterministic flow.

Use Case :

- Processing an order step-by-step with validations.

Diagram :



Use Case Example :

Processing an order step-by-step: Validate -> Charge -> Send Confirmation.

⑤ Runtime Monitoring

Runtime monitoring tracks performance, errors, latency, and resource usage during execution.

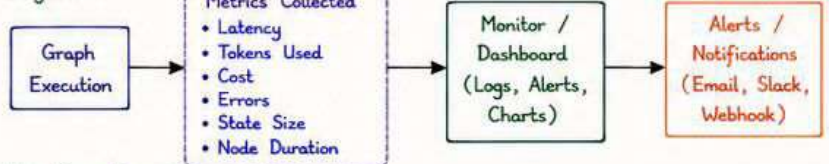
Key Points :

- Monitor latency, tokens, cost, errors.
- Track node-level metrics.
- Set alerts for failures or slow performance.
- Essential for production applications.

Use Case :

- Production monitoring and alerting for agent systems.

Diagram :



Use Case Example :

Alert if any node fails, or if response time > 5 seconds.
Dashboard shows tokens used and cost per request.



Takeaway : Streaming improves user experience. Async improves performance. Monitoring ensures reliability. Choose the right execution model for the right use case.



LangGraph with Agentic AI

Module 10 : Persistence

Save Progress, Recover Gracefully, Execute Reliably

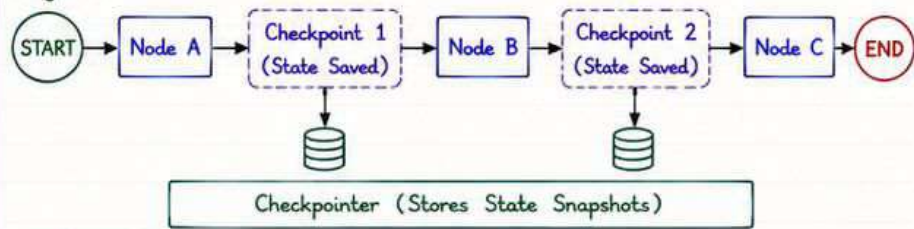
① Checkpointers

Checkpointers save the state of the graph at certain points so execution can be resumed later if needed.

Key Points :

- Capture graph state at checkpoints.
- Include node state, memory, and context.
- Enable pause, resume, and recovery.
- Essential for long-running workflows.

Diagram :



Use Case Example :

A document processing flow saves state after each major step. If interrupted, it resumes from the last checkpoint.

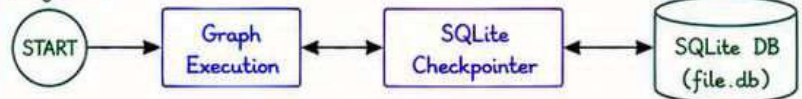
② SQLite Checkpointer

Stores checkpoints in a local SQLite database.

Key Points :

- Lightweight and file-based.
- Easy to set up and use.
- Good for development and small apps.
- Stores state as serialized JSON/BLOB.

Diagram :



Use Case Example :

A local chatbot app uses SQLite checkpointer to save conversation state and resume if the app restarts.

③ PostgreSQL Checkpointer

Stores checkpoints in a PostgreSQL database.

Key Points :

- Robust and scalable.
- Supports concurrent access.
- Ideal for multi-user and production systems.
- Better durability and backup options.

Diagram :



Use Case Example :

A SaaS chatform with many users stores checkpoints in PostgreSQL for reliability, scaling and high availability.

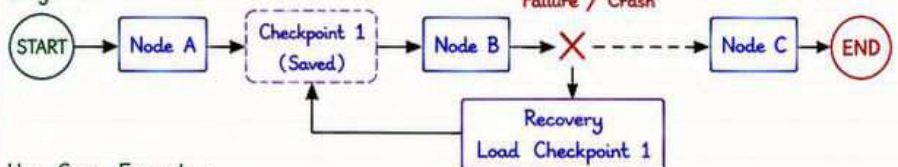
④ Recovery After Failures

If a failure occurs, the system can recover from the last saved checkpoint.

Key Points :

- Detect failures or crashes.
- Load the latest valid checkpoint.
- Resume execution from that point.
- Prevents data loss and rework.

Diagram :



Use Case Example :

If the system crashes during report generation, it restarts from the last checkpoint instead of starting all over.

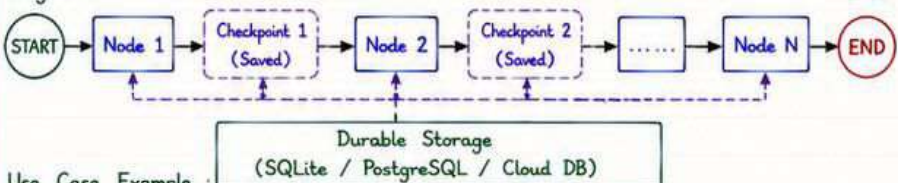
⑤ Durable Execution

Ensures the graph can run reliably for long periods without losing progress.

Key Points :

- Frequent and reliable checkpoints.
- Persistent storage with backups.
- Exactly-once / at-least-once guarantees.
- Suitable for long-running and critical workflows.

Diagram :



Use Case Example :

A data pipeline runs for hours/days with checkpoints saved continuously, ensuring no data is lost even if there are crashes or restarts.



Takeaway : Persistence makes LangGraph workflows reliable and production-ready. Choose the right checkpointer, save state frequently, and recover gracefully from any failure.



* LangGraph with Agentic AI *

Module 11 : Debugging & Observability (Part 1)

See Clearly, Understand Deeply.
Fix Faster.

x-----x

① Graph Visualization

Graph visualization helps us understand the structure of the LangGraph, nodes, edges and flow before and during execution.

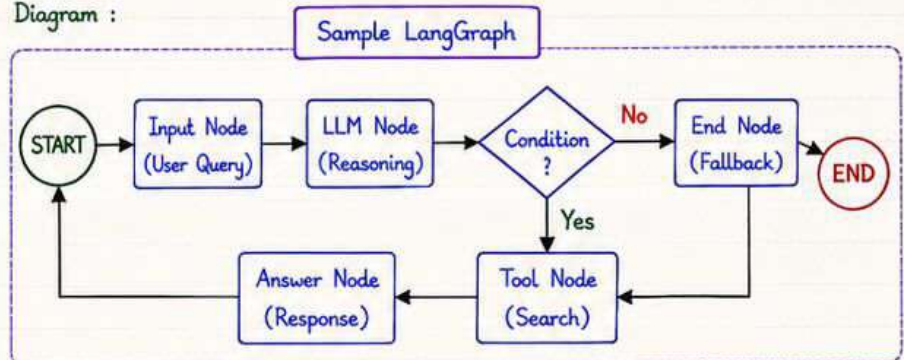
Key Points :

- Visual view of nodes and connections.
- Helps in understanding complex graphs.
- Detects missing or incorrect edges.
- Available before running and after changes.

Use Case :

- Visualize an agent workflow before deploying to ensure the logic is correct.

Diagram :



Use Case Example :

A developer visualizes the graph to confirm that after the LLM decision, the tool is called only when condition is true.

② Execution Tracing

Tracing shows the step-by-step execution path, inputs, outputs, and state changes for each node.

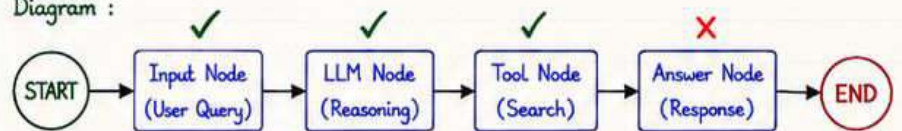
Key Points :

- Records every node execution.
- Shows input, output, and state diffs.
- Helps in finding where things went wrong.
- Useful for performance and logic analysis.

Use Case :

- Trace a failed response to find which node produced an incorrect output.

Diagram :



Execution Trace (Example)

Step	Node	Input (Summary)	Output (Summary)	Status	Time (ms)
1	Input Node	What is LangGraph?	User Query Received	✓	12
2	LLM Node	User Query	Plan : Call Search	✓	1450
3	Tool Node	Search : LangGraph	Top 3 Results	✓	820
4	Answer Node	Results + Plan	(Error) Timeout	✗	1030

Use Case Example :

Tracing shows that the Answer Node timed out. Developer can now optimize that node or increase timeout.

③ Logging

Logging captures important events, errors, warnings and info during execution for monitoring and debugging.

Key Points :

- Different log levels: INFO, DEBUG, WARN, ERROR.
- Logs can be saved to console, file or external systems.
- Helps in production monitoring.
- Enables alerts and issue tracking.

Use Case :

- Log errors and performance metrics in production to quickly detect issues.

Diagram :

2025-08-05 12:10:01	INFO	Input Node	Received query: What is LangGraph?
2025-08-05 12:10:01	DEBUG	LLM Node	Prompt tokens: 120, Temp: 0.7
2025-08-05 12:10:02	INFO	LLM Node	Plan: Call Search Tool
2025-08-05 12:10:03	INFO	Tool Node	Search executed. Results: 3
2025-08-05 12:10:05	WARN	Answer Node	Response time high: 5200ms
2025-08-05 12:10:06	ERROR	Answer Node	Timeout after 5s

Use Case Example :

Logs help the team identify that Answer Node frequently times out in production and trigger an alert.



Takeaway : Visualization helps you see, tracing helps you follow, and logging helps you know what's happening.



* LangGraph with Agentic AI *

Module 11 : Debugging & Observability (Part 2)

See Clearly. Understand Deeply.
Fix Faster.

④ Debugging Nodes

Debugging nodes help inspect, validate and print data at any point in the graph to find issues quickly.

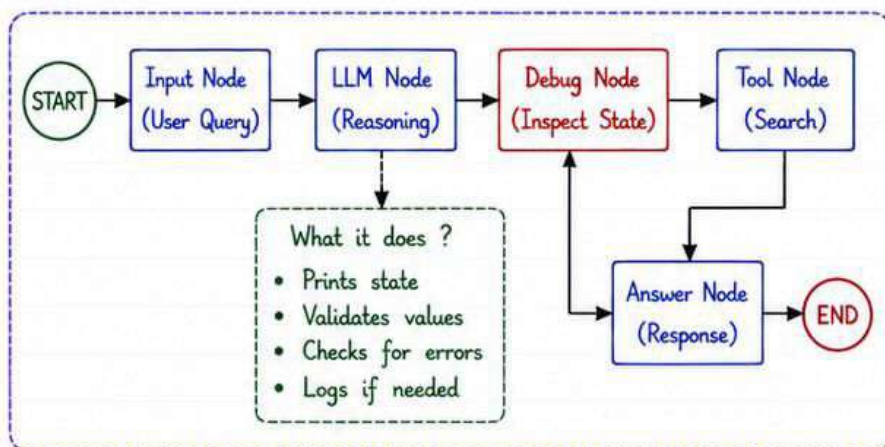
Key Points :

- Inspect inputs, outputs and state.
- Validate data and catch edge cases.
- Print, log or assert values.
- Helps isolate and fix bugs faster.

Use Case :

- Add a Debug node to check what data is flowing between complex steps.

Diagram :



Use Case Example :

Developer adds Debug Node after LLM to print the plan generated. Helps to find why tool is not getting correct query.

⑤ LangSmith Integration

LangSmith is an observability platform that helps monitor, debug and evaluate LLM applications built with LangGraph.

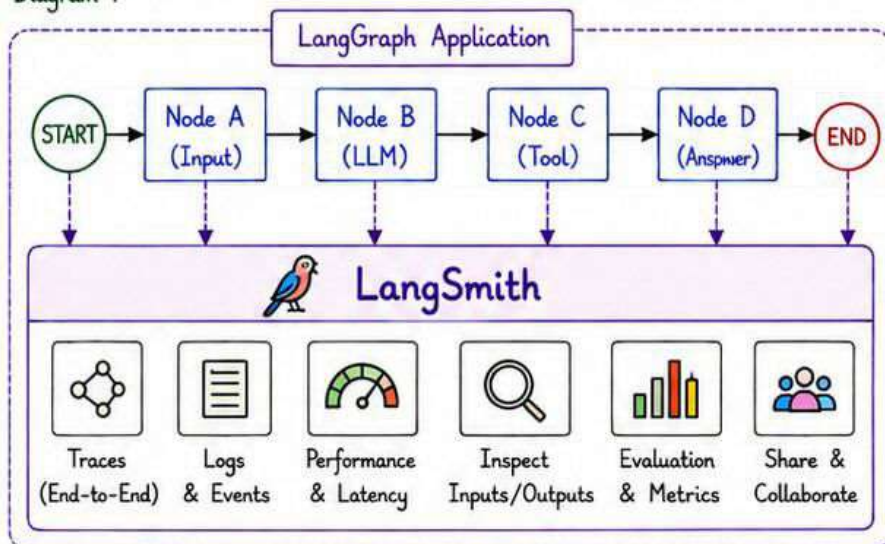
Key Points :

- End-to-end tracing of graph executions.
- Visualize runs, nodes and timelines.
- Inspect inputs, outputs, errors and latencies.
- Evaluate and compare model outputs.
- Share, annotate and collaborate.
- Works great with LangGraph out-of-the-box.

Use Case :

- Monitor a production Agent workflow, debug slow nodes and evaluate responses.

Diagram :



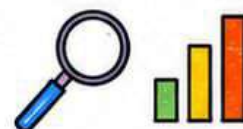
Use Case Example :

- ✓ You deploy a customer support agent.
- ✓ LangSmith shows which node is slow (e.g., Tool Node).
- ✓ You inspect inputs/outputs and find the wrong query format.
- ✓ Fix the node and compare performance before vs after.



Takeaway :

- Use Debug nodes to inspect and fix issues in the flow.
- Use LangSmith to observe, analyze and improve your LangGraph applications.





LangGraph Interview Questions

Top 50 (Easy to Advanced)



TOP 25 INTERVIEW QUESTIONS (EASY TO INTERMEDIATE)

- 1 What is LangGraph, and why is it used for building Agentic AI applications?
- 2 What is the difference between LangGraph and LangChain?
- 3 What are State, Nodes, and Edges in LangGraph?
- 4 How does state management work in LangGraph?
- 5 What is a StateGraph in LangGraph?
- 6 How do you create a basic LangGraph workflow?
- 7 What is the difference between normal edges and conditional edges?
- 8 How do you implement conditional routing in LangGraph?
- 9 How do you implement tool calling in LangGraph?
- 10 How do you connect an LLM with tools in LangGraph?
- 11 How do you build a ReAct-style agent using LangGraph?
- 12 How do you implement loops in LangGraph?
- 13 How do you prevent an agent from entering an infinite loop?
- 14 How do you handle errors and failed tool calls in LangGraph?
- 15 How do you implement memory in LangGraph?
- 16 What is checkpointing in LangGraph, and why is it important?
- 17 How do you maintain conversation history across multiple user interactions?
- 18 How do you implement Human-in-the-Loop (HITL) in LangGraph?
- 19 How do you build a multi-agent system using LangGraph?
- 20 How do agents communicate with each other in a LangGraph workflow?
- 21 How do you integrate RAG with LangGraph?
- 22 How do you integrate MCP tools with LangGraph?
- 23 How do you debug and monitor a LangGraph application in production?
- 24 How would you optimize a LangGraph agent for latency, token usage, and cost?
- 25 How would you design and deploy a production-ready LangGraph Agentic AI application?



Stateful Workflows



Tool Calling



Memory & Checkpointing



Multi-Agent Systems



Production Ready



TOP 25 ADVANCED INTERVIEW QUESTIONS (ADVANCED LEVEL)

- 1 How would you design a production-grade LangGraph architecture for a complex Agentic AI application?
- 2 How does state management work in LangGraph, and how would you design state for a multi-agent workflow?
- 3 What is the difference between StateGraph, nodes, edges, and conditional edges, and how do they work together?
- 4 How would you implement dynamic routing when the next node depends on the LLM's decision?
- 5 How would you design a LangGraph workflow with multiple agents having different responsibilities?
- 6 How would you implement supervisor-based multi-agent architecture using LangGraph?
- 7 How would you prevent infinite loops in a LangGraph agent?
- 8 How would you implement retry, timeout, fallback, and error recovery for failed tools or nodes?
- 9 How would you handle a situation where an LLM repeatedly selects the wrong tool?
- 10 How would you implement Human-in-the-Loop (HITL) in a LangGraph workflow for high-risk operations?
- 11 How does checkpointing work in LangGraph, and how would you use it for long-running workflows?
- 12 How would you implement short-term and long-term memory in a LangGraph Agent?
- 13 How would you persist and restore an agent's state after a server restart or failure?
- 14 How would you design a LangGraph agent capable of parallel tool execution?
- 15 How would you optimize a LangGraph workflow to reduce end-to-end latency?
- 16 How would you optimize a LangGraph agent to reduce LLM token consumption and cost?
- 17 How would you integrate RAG into LangGraph and dynamically decide whether retrieval is required?
- 18 How would you integrate MCP servers and MCP tools with LangGraph?
- 19 How would you implement tool selection, tool validation, and tool-result handling in a production LangGraph agent?
- 20 How would you implement guardrails and security around a LangGraph agent that can execute external tools?
- 21 How would you evaluate a LangGraph Agent's planning, tool selection, task completion, reliability, and final answer quality?
- 22 How would you debug a production LangGraph workflow when the agent produces an incorrect answer?
- 23 How would you implement observability and tracing for LangGraph—including node execution, tool calls, latency, tokens, errors, and intermediate states?
- 24 How would you design LangGraph for scalability and concurrent users in a production environment?
- 25 A production LangGraph agent suddenly has higher latency, increased token cost, more tool failures, and lower answer quality. How would you investigate the problem and identify the root cause?



Build Smarter AI Agents



Orchestrate Complex Workflows



Scale with Confidence



Secure, Reliable & Observed



Collaborate Intelligently